TraCast

Traffic speed forecasting on METR-LA

ivanimmanuel-dev / TraCast | Research brief, November 2026

What I wanted to know

Do GRU and LSTM models beat simple baselines? Does the road graph help? Does knowing the time of day help? And does federated training lose anything compared with training on all the data in one place?

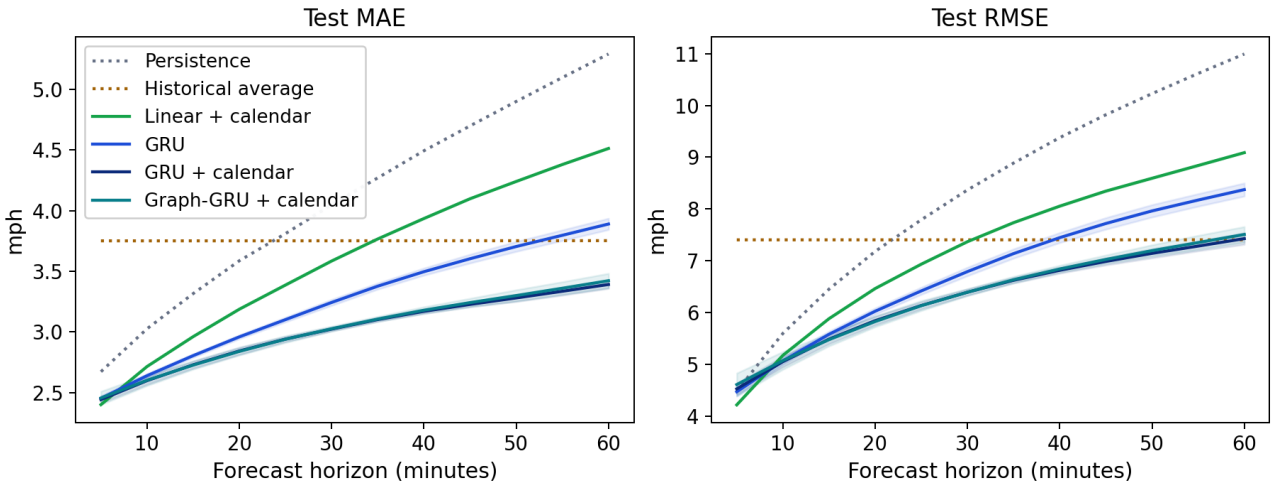
What I found

- **The clock helps more than anything else I tried.** Adding the time of day and a weekend flag took the GRU from 3.255 to 3.007 mph MAE, a change of -0.249 mph (95% CI -0.324 to -0.181), and at the 60-minute horizon from 3.889 to 3.391. Training longer helped too, but less: five epochs gave 3.409.
- **The road graph doesn't give a reliable gain.** The neighbor averages help a little without the clock (MAE -0.038 mph) and hurt a little with it (+0.031). The best three models are within 0.031 mph of each other, so I can't rank them.
- **Everything beats persistence (4.100).** The historical average (3.751) and a linear model (3.622) land in between, and the GRUs are well ahead of both. (At 15 minutes persistence is still ahead of the historical average.)
- **Federated averaging is about as good as training each client alone.** At 5 rounds with balanced groups, federated averaging gets 3.774 mph, local-only 3.796 and centralized 3.750. With congestion-grouped clients it is 3.836 against 3.769.

Results

Model	MAE (SD)	15 min	60 min
Persistence	4.100	3.316	5.290
Historical average	3.751	3.751	3.752
Linear	3.622 (0.001)	2.967	4.572
GRU	3.255 (0.026)	2.805	3.889
LSTM	3.271 (0.013)	2.845	3.885
Graph-GRU	3.193 (0.038)	2.779	3.765
GRU + calendar	3.007 (0.025)	2.730	3.391
Graph-GRU + calendar	3.016 (0.031)	2.727	3.422

Test MAE in mph, 16 sensors, 24 test days; learned models average 5 seeds.



Setup

METR-LA has 34,272 five-minute readings from 207 sensors (March 1 to June 27, 2012). I use 16 of them, picked by walking the DCRNN distance graph. Rows are split in time: 70% train, 10% validation, 20% test. A model sees one hour and predicts the next hour. Zero and missing readings are left out of the loss and the scores, and normalization uses training data only.

Graph-GRU gives each sensor its own speed plus the average of the sensors upstream and downstream of it, passes that through a small shared layer and feeds all 16 sensors into a GRU.



"Self only" zeroes the two averages. Every model reads all 16 sensors, so that tests the graph-weighted averages, not whether neighbors matter.

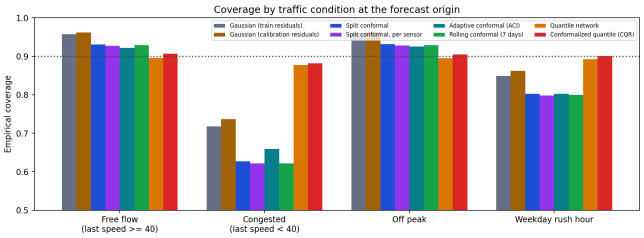
Federated learning

Three simulated clients each hold four of twelve sensors and train the same small GRU with Flower; the server averages the weights. I compare it with training on all the data together and with each client training alone. At 5 rounds with balanced groups, federated averaging gets 3.774 mph, local-only 3.796 and centralized 3.750. With congestion-grouped clients it is 3.836 against 3.769. The gaps are about the size of the seed spread, so in this setup training alone is about as good as federating.

Going from 1 to 5 rounds cuts balanced MAE by 0.273 mph and sends 4 times as many model-weight bytes. The byte counts cover weights sent for training, validation and the final test; message overhead isn't counted.

Bigger networks, intervals and drift

- **In the shared models the graph helps, at 5 of 5 sizes.** Adding the neighbor averages changes MAE by -0.057 to -0.029 mph. Without them a shared model sees nothing about its neighbors, so this is a clean test of the graph, which the dense models can't give.
- **Sharing the weights wins once the network is big.** At 128 and 207 sensors the shared GRU beats the dense GRU by 0.157 to 0.242 mph (the interval excludes zero), and at 16, 32 and 64 they are level. It does that with 4,044 parameters instead of 105,396.
- **In a shared model, telling the sensors apart matters more than the graph.** A learned 4-number vector per sensor changes test MAE by -0.208 to -0.107 mph at 16 and 64 sensors, the graph's neighbor averages by -0.055 to -0.050 and an LSTM instead of a GRU by -0.030 to -0.020 (negative is better).
- **Prediction intervals that are right on average can fail when it matters.** At a nominal 90%, split conformal covers 90.5% of the test speeds, but only 63% when the last speed was below 40 mph (against 93% in free flow). A quantile network with a conformal correction holds 88% there at the same average width.
- **Adapting a deployed model helps a little, and cheap adaptation can hurt.** Fine-tuning the output layer daily improves MAE by 0.031 mph and fine-tuning the whole network weekly by 0.042 (for a model trained on only the first 40% of the timeline: 0.072 and 0.188), at 3 s and 27 s for the 24 test days. Averaging past errors into the forecast was worse than doing nothing (+0.094 mph at best).



What this doesn't cover

One 16-sensor subset, one 24-day test window, no hyperparameter search, and a federated task that is univariate and simulated on one machine. The later experiments have their own limits in their reports.

github.com/ivanimmanuel-dev/TraCast
The technical report has the details; DATA.md has the data sources.

References: Li et al., ICLR 2018 (METR-LA, DCRNN); Cho et al., EMNLP 2014 (GRU); Hochreiter and Schmidhuber, 1997 (LSTM); McMahan et al., AISTATS 2017 (FedAvg); Flower 1.42.0.